

Hypothesis-Driven Memory-Augmented Bee Colony Framework: A Conceptual Architecture for Lifelong Swarm Optimization

Lei Huang
Independent Researcher
Chongqing, China
2644901977@qq.com

Version 1.0 — April 27, 2026

Abstract

Most population-based swarm optimizers are designed as episodic search procedures: when a new optimization task begins, the algorithm typically restarts from a randomly initialized population and discards most structural experience accumulated in previous tasks. This design is effective for isolated black-box optimization, but it is poorly matched to sequential optimization settings in which related tasks appear over time, misleading priors may be discovered, and historical search experience could be selectively reused. This technical report proposes the Hypothesis-Driven Memory-Augmented Bee Colony Framework (HD-MABC), a conceptual architecture that augments artificial bee colony optimization with selective memory, cognitive conflict detection, hypothesis generation, and bounded asynchronous branch exploration. The framework treats long-term memory not as an infallible archive, but as a revisable set of region-level hypotheses whose reliability must be tested against new evidence. This report defines the architecture, motivates the cognitive design analogy, specifies the key data objects and operational loop, and states two falsifiable scientific conjectures: the memory acceleration effect and the hypothesis leap effect. No empirical superiority claim is made in this version. The purpose of the report is to establish a precise research object and a reproducible implementation target for subsequent experiments.

Keywords: artificial bee colony algorithm; swarm intelligence; lifelong optimization; memory-augmented optimization; hypothesis-driven search; cognitive conflict.

1 Introduction

Swarm intelligence algorithms such as artificial bee colony optimization, particle swarm optimization, and ant colony optimization have become widely used tools for black-box optimization because they combine simple local update rules with global population-level exploration [5, 8–10]. Related metaheuristics such as simulated annealing and differential evolution also demonstrate the value of derivative-free adaptive search [11, 15]. These algorithms are especially attractive when gradients are unavailable, objective functions are noisy, or the search landscape is multimodal. Their success, however, has mostly been studied under an episodic assumption: each benchmark run is treated as a fresh and independent problem instance.

This assumption hides a fundamental limitation shared by many mainstream swarm optimizers. Each optimization episode usually starts from scratch, with little structured reuse of historical search experience. Even if the optimizer repeatedly solves tasks drawn from the same problem family, the search procedure often behaves as if no previous task had been observed. In this sense, many swarm optimizers exhibit a form of algorithmic amnesia: they may adapt within a single run, but they do not maintain a durable, inspectable, and task-relevant memory that improves future runs.

The limitation matters in realistic sequential optimization scenarios. In AutoML, scientific experiment design, logistics scheduling, simulator calibration, and engineering design, optimization tasks often arrive as streams rather than isolated benchmarks. Consecutive tasks may share latent structure, such as similar high-quality regions, recurring constraints, or repeated deceptive patterns. A memoryless optimizer wastes evaluations rediscovering such structure. It also lacks a principled way to revise a misleading prior when later evidence contradicts earlier experience.

This report proposes the Hypothesis-Driven Memory-Augmented Bee Colony Framework (HD-MABC), an architecture inspired by the functional division between short-term working memory, long-term structured memory, and conflict-sensitive control in human cognition. The central idea is not merely to store good solutions. Rather, the framework treats optimization as a lifelong process in which an algorithm selectively remembers useful evidence, forgets stale or disproven associations, detects conflict between expectation and observation, generates competing hypotheses, and allocates bounded search branches to test those hypotheses.

The current version is intentionally a conceptual technical report. It contributes:

1. a four-layer architecture for memory-augmented bee colony optimization;
2. a mapping between cognitive memory functions and optimization modules;
3. three key mechanisms: selective memory and forgetting, cognitive conflict detection with hypothesis generation, and bounded asynchronous branch exploration;
4. an implementation-oriented specification of core data objects and an operational loop;
5. two falsifiable conjectures and a future experimental protocol that can guide reproducible validation.

1.1 Research Questions

The framework is organized around three research questions.

1. **Can a swarm optimizer learn across related tasks?** If consecutive tasks share latent structure, an optimizer with selective memory should reduce the number of evaluations needed to reach a target quality level.
2. **Can a memory-augmented optimizer avoid becoming trapped by its own memory?** If a historical rule becomes misleading, the optimizer should detect conflict and allocate resources to test alternative explanations.
3. **Can hypothesis-driven branching create a measurable convergence signature?** If the mechanism works, deceptive landscapes should exhibit a characteristic transition: stagnation under a false prior followed by rapid improvement after conflict-triggered re-exploration.

These questions deliberately separate architecture from empirical validation. The present report defines the mechanism and the expected signatures. It does not report that the signatures have already been observed.

1.2 Positioning and Claim Boundaries

The idea of using memory in evolutionary and swarm computation is not new. Dynamic optimization has studied memory-enhanced evolutionary algorithms, immigrant schemes, archives, and adaptive mechanisms for changing environments [3, 16]. Classical evolutionary computation also has a long history of population-level adaptation and implicit information reuse [6]. The intended distinction is narrower: the framework requires memory to be structured, revisable, and

connected to a conflict-testing mechanism. Existing memory-assisted optimizers typically satisfy at most one of these three conditions simultaneously.

The framework combines these three requirements in a single architecture. First, memory is not merely an archive of elite solutions; it is a structured belief system that can include positive, negative, uncertain, and contradicted region-level knowledge. Second, the optimizer is designed to detect when its own memory becomes suspicious. Third, detected conflicts are converted into explicit, bounded hypotheses that compete for evaluation budget. The proposed contribution is the integration of memory, self-questioning, and hypothesis-driven branch testing in an artificial bee colony setting.

This boundary is important for future empirical work. A fair evaluation should compare the proposed method not only with standard ABC but also with memory-based evolutionary algorithms, dynamic-environment variants, and other adaptive swarm optimizers. If the proposed method only outperforms a naive baseline, the scientific claim would be weak. The stronger claim, to be tested later, is that conflict-driven hypothesis testing adds value beyond ordinary memory reuse.

1.3 Relationship to Existing Work

The framework is closest to three lines of work. The first is artificial bee colony optimization, where search is organized around employed bees, onlooker bees, and scout bees [8, 9]. ABC already contains local adaptation within a run: poor food sources are abandoned, better food sources attract more search effort, and scouts reintroduce diversity. However, standard ABC does not normally preserve an explicit long-term model of which regions were useful or misleading across a stream of tasks.

The second line is evolutionary optimization in dynamic or uncertain environments. Prior work has explored memory archives, immigrant schemes, environmental change detection, and diversity-preserving strategies [3, 4, 7, 14, 16]. This literature is highly relevant because it studies how population-based search can respond when the objective changes. The proposed framework differs in emphasis: it is not only designed to react to external environmental change, but also to inspect and revise the algorithm’s own internal beliefs when they conflict with new observations.

The third line is cognitive and complementary learning systems. Working memory and long-term memory serve different computational roles, and conflict monitoring can redirect behavior when prediction and observation diverge [1, 2, 12, 13]. The HD-MABC does not claim biological fidelity. Instead, it uses this division as an engineering principle: short-term evidence should be filtered, long-term rules should be consolidated and revised, and conflict should trigger controlled exploration rather than blind persistence.

The resulting design goal is not to replace existing swarm optimizers, but to define a memory-and-conflict layer that can be attached to an ABC-style execution engine and evaluated through ablation studies.

The report does not present implementation results, statistical tests, or claims of superiority over existing algorithms. Those are future empirical tasks. The narrower purpose is to define the research object with enough precision that it can be implemented, criticized, and tested.

2 Core Insight

Human cognition does not treat every observation equally. Working memory temporarily holds task-relevant information, filters distractors, and supports goal-directed control. Long-term memory consolidates recurring patterns and makes them available for future reasoning. When an expectation fails, cognitive control can redirect attention, revise assumptions, or trigger exploratory behavior. The biological details are complex, but the high-level division provides a useful design metaphor for sequential optimization [1, 12, 13].

The design analogy is useful because optimization under a sequence of related tasks creates the same engineering tension as continual learning. The optimizer must preserve reusable structure, but it must not preserve every observation. It must exploit a useful prior, but it must not become unable to revise that prior. It must react to surprising evidence, but it must not spend unlimited evaluations on every anomaly. These tensions motivate the separation between a fast short-term buffer, a slower long-term knowledge base, and a conflict-sensitive controller.

The proposed framework maps this division to optimization in the following way.

Table 1: Cognitive analogy used by the HD-MABC architecture.

Cognitive component	Functional role	Optimization module
Prefrontal memory	Short-term retention, information filtering, task-directed selection	Short-term memory buffer that labels recent samples as positive evidence, negative evidence, or noise
Hippocampal memory system	Long-term consolidation, relational structure, pattern reuse	Structured long-term knowledge base storing reusable region-level rules and task-family associations
Conflict-sensitive control	Attention triggered by mismatch between expectation and outcome	Cognitive conflict detector followed by hypothesis generation and branch exploration

This analogy is used cautiously. The framework does not claim to be a neuroscientific model. Instead, it borrows a useful computational principle: an optimizer should not only exploit current samples, but should also manage the reliability of its own experience. A memory item can be helpful, irrelevant, stale, or actively misleading. Therefore, memory must be selective, revisable, and connected to a mechanism for testing alternative explanations.

2.1 Why Memory Alone Is Insufficient

Adding memory to an optimizer can make performance better or worse. If memory is accurate and task-relevant, it can accelerate search. If memory is stale, overgeneralized, or transferred to an incompatible task, it can bias the search away from the optimum. This risk is especially severe in sequential optimization because the same problem family may contain both stable regularities and deceptive exceptions. A long-term rule such as “this quadrant is usually low-potential” may be useful for most tasks but catastrophic for a task whose optimum lies exactly there.

For this reason, the framework treats memory as a fallible hypothesis rather than as a fixed truth. The optimizer should ask: Which previous observations support this memory item? How often has it been useful? Under what task conditions did it fail? Does the current sample contradict it strongly enough to justify re-exploration? These questions convert memory from passive storage into an active control object.

2.2 From Search Heuristic to Scientific Object

The framework is also intended to make the algorithm easier to study scientifically. Many metaheuristics combine several operators and then report aggregate performance. Such designs can work, but they make it hard to identify why an improvement occurred. By separating memory labeling, conflict detection, hypothesis generation, and branch testing, the HD-MABC creates modules that can be individually removed in ablation studies. If the hypothesis leap effect disappears when conflict detection is removed, the mechanism has a clearer explanatory role.

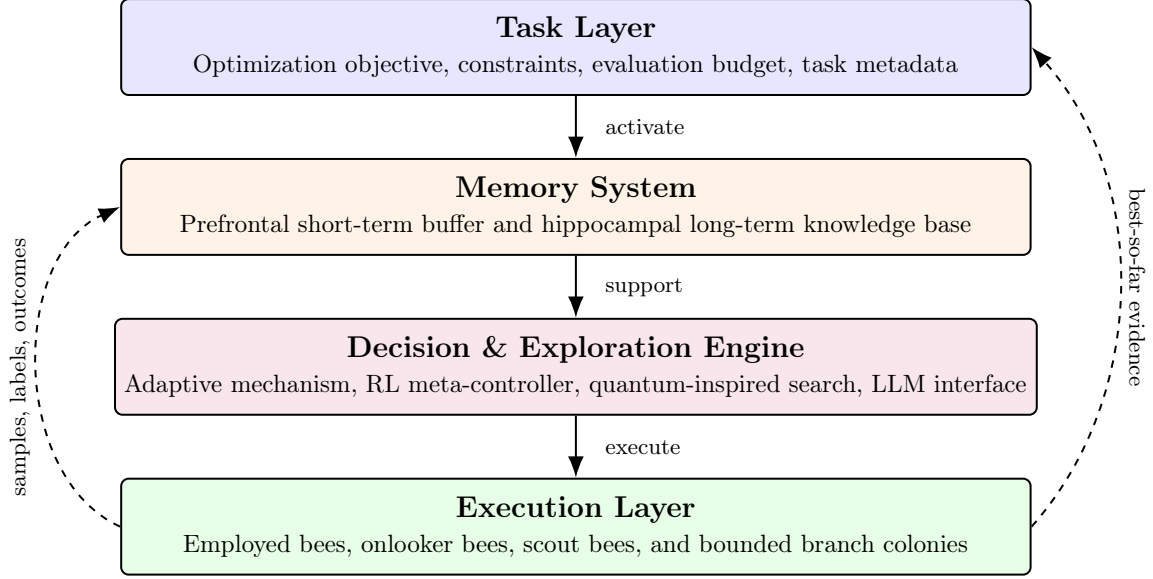


Figure 1: Four-layer architecture of the HD-MABC system. The memory system is placed between task specification and search execution so that past experience can influence the current run, while current evidence can update or invalidate the memory.

This modularity also reduces the risk of vague metaphor. The biological analogy is only useful if each analogy maps to a measurable computational operation. In the proposed framework, prefrontal filtering corresponds to short-term evidence labeling; hippocampal consolidation corresponds to region-rule creation and confidence updates; conflict monitoring corresponds to thresholded mismatch detection; hypothesis testing corresponds to branch allocation and arbitration. Each mapping can be implemented, removed, or replaced.

The core insight is: optimization algorithms should not be defined as memoryless search procedures. A genuine lifelong optimizer should possess selective memory, cognitive curiosity, and self-correction through hypothesis verification.

3 Architecture

Figure 1 shows the four-layer architecture of the proposed framework. The architecture is intentionally modular. A minimal prototype may implement only simple interval-based memory and a standard artificial bee colony execution layer. Later versions can replace individual modules with stronger variants, such as learned region embeddings, reinforcement-learning-based meta-control, or language-model-assisted experiment planning.

3.1 Task Layer

The task layer defines the optimization problem currently being solved. Its input includes the objective function, domain bounds, constraints, evaluation budget, termination criteria, and optional metadata describing the task family. Its output is not a solution; instead, it emits a task context that activates relevant memories and configures the search engine.

For a minimal black-box setting, the task context may contain only dimensionality, bounds, and budget. In richer settings, it may include feature descriptors such as variable types, constraint categories, known symmetries, or similarity to previous tasks. The framework does not require these descriptors, but it can exploit them when available.

3.2 Memory System

The memory system has two components. The prefrontal module is a short-term memory buffer for the current run. It stores recently sampled solutions, objective values, local improvement attempts, and labels indicating whether the evidence is positive, negative, or irrelevant. This buffer is intentionally volatile: it is optimized for fast filtering and immediate decision support.

The hippocampus module is a long-term structured knowledge base. It stores reusable rules learned across tasks. In a minimal prototype, a rule can be represented as an interval or hyper-rectangle associated with a region label, such as high-potential, low-potential, uncertain, or contradicted. In stronger implementations, the knowledge base may store distributions, embeddings, surrogate models, or task-conditioned region statistics. The important requirement is that long-term memory must be inspectable and revisable. It should not be an opaque archive of all previous samples.

3.3 Decision and Exploration Engine

The decision and exploration engine translates task context and memory signals into search actions. It decides which regions to exploit, which regions to avoid, which stale memory items to test, and when to allocate a bounded branch for an alternative hypothesis. In the full architecture, this layer may include four submodules:

1. an adaptive mechanism that adjusts local search radius, scout probability, memory trust, and exploration pressure;
2. a reinforcement-learning meta-controller that learns high-level policies for selecting operators or allocating budgets across branches;
3. a quantum-inspired search component that maintains diverse candidate states or probabilistic superpositions over search regions;
4. a language-model interface that helps summarize memory, generate human-readable hypotheses, or document experiment traces.

These submodules are optional extensions. A first prototype should not depend on all of them. The core requirement is that the engine can convert conflicts into testable hypotheses and can allocate limited evaluations to those hypotheses without derailing the main search.

3.4 Execution Layer

The execution layer contains the bee-colony search entities. In a standard artificial bee colony algorithm, employed bees explore around current food sources, onlooker bees select food sources based on quality, and scout bees replace exhausted sources with new random samples [8, 9]. The HD-MABC execution layer preserves this operational structure but adds two responsibilities.

First, each search action returns evidence to the memory system. A sample is not merely a candidate solution; it is also an observation about the reliability of a region, rule, or prior. Second, when the decision engine creates competing hypotheses, the execution layer can launch bounded branch colonies. Each branch receives a subgoal, a limited evaluation budget, and a reporting protocol. Branches that fail to improve are terminated quickly, while promising branches can update the main colony or revise long-term memory.

3.5 Core Data Objects

The architecture can be implemented with a small set of explicit data objects. Making these objects explicit helps prevent the framework from becoming an informal collection of heuristics.

Table 2: Minimal data objects required by a first prototype.

Object	Role
Task context	Objective handle, bounds, dimension, constraints, evaluation budget, task identifier, and optional task-family metadata.
Sample record	Candidate vector, objective value, evaluation index, source bee or branch, and local improvement status.
Short-term memory item	Recent sample record plus evidence label, relevance score, and contradiction score.
Long-term memory rule	Region descriptor, region label, quality estimate, confidence, provenance, usage count, and contradiction count.
Conflict event	Trigger type, affected memory rule, current evidence, severity score, and generated hypotheses.
Branch report	Hypothesis identifier, branch initialization region, budget consumed, best improvement, and decision recommendation.

The first prototype should avoid overengineering these objects. Region descriptors can be simple intervals or hyper-rectangles. Confidence can be a scalar in $[0, 1]$. Contradiction severity can begin as a thresholded deviation between expected and observed fitness. The point is not to design the final representation immediately; it is to preserve the logical separation between evidence, memory, conflict, and action.

3.6 Information Flow

During a run, the task layer initializes the task context and queries long-term memory for relevant rules. The memory system returns priors such as high-potential regions, low-potential regions, and uncertain regions. The decision engine uses these priors to bias the initial bee population or to modify scout behavior. The execution layer then samples candidate solutions and returns sample records. These records update the short-term buffer. Periodically, the buffer consolidates reliable evidence into long-term memory and checks whether observations contradict existing rules.

This feedback loop is the central architectural difference from ordinary episodic ABC. In standard ABC, a poor region may be abandoned within a run, but that fact is not normally turned into a persistent, inspectable rule. In the HD-MABC, the abandonment decision becomes evidence. Later tasks can use it, challenge it, or erase it.

3.7 State Variables and Interfaces

Let the current task be $T_k = (f_k, \Omega_k, B_k)$, where f_k is the objective function, $\Omega_k \subset \mathbb{R}^d$ is the bounded search domain, and B_k is the evaluation budget. The colony state at evaluation step n is

$$\mathcal{P}_n = \{(x_i, f_k(x_i), \ell_i, a_i)\}_{i=1}^N, \quad (1)$$

where x_i is a candidate solution, ℓ_i is its trial counter or abandonment state, and a_i records the action source that generated it. The short-term memory buffer is denoted by $\mathcal{M}_k^S$, and the long-term memory base is denoted by $\mathcal{M}^L$.

The task layer calls a memory retrieval interface

$$\text{Retrieve}(\mathcal{M}^L, T_k) \rightarrow \Pi_k, \quad (2)$$

where Π_k is a set of priors over regions. A prior can encourage initialization, discourage scout placement, or mark a region as uncertain and worth testing. During search, the execution layer

calls an evidence interface

$$\text{UpdateShortTerm}(\mathcal{M}_k^S, x, f_k(x), a), \quad (3)$$

and the memory layer periodically calls

$$\text{Consolidate}(\mathcal{M}_k^S, \mathcal{M}^L) \rightarrow \mathcal{M}^L. \quad (4)$$

Conflict detection uses

$$\text{DetectConflict}(\mathcal{M}_k^S, \mathcal{M}^L, \Pi_k) \rightarrow \mathcal{C}_k, \quad (5)$$

where $\mathcal{C}_k$ is a set of conflict events. The decision engine converts each conflict into one or more hypotheses and passes bounded search directives back to the execution layer. These explicit interfaces are intended to make the framework implementable without committing to a single memory representation.

4 Key Mechanisms

The proposed architecture depends on three mechanisms. They are described here at the level needed to guide implementation. Precise parameter values are left for future experimental work.

4.1 Selective Memory and Forgetting

The first mechanism is selective memory. A naive memory-augmented optimizer might store every historical solution, but that quickly becomes inefficient and unreliable. The HD-MABC instead stores evidence with labels and validity conditions. A memory record can be represented as

$$m = (r, y, q, c, t, s), \quad (6)$$

where r is a region descriptor, y is a label, q is an estimated quality score, c is a confidence value, t is a timestamp or task index, and s stores provenance such as the originating task or branch.

The short-term buffer classifies current-run evidence into three broad types:

1. **Positive knowledge:** samples, regions, or operators that produced high-quality food sources or consistent improvement.
2. **Negative knowledge:** sampled regions that were tested sufficiently and appeared unproductive under the current task.
3. **Noise or garbage information:** observations that are weakly related to the objective, are dominated by measurement noise, or cannot be reliably connected to future decisions.

Two criteria guide filtering. The first is task relevance: evidence should be retained only if it helps choose future actions under the active objective and constraints. The second is contradiction strength: evidence becomes more important when it confirms, weakens, or directly contradicts long-term memory. A mediocre sample in an unimportant region may be forgotten quickly; a surprisingly strong sample inside a historically low-potential region should be retained because it may indicate a false prior.

Forgetting is not a failure mode; it is a necessary control mechanism. A memory item should decay when it is not reused, when its associated region has repeatedly failed on later tasks, or when a stronger rule explains the same observations. Equation (7) defines a simple forgetting score that combines age, retrieval frequency, confidence decay, and contradiction count:

$$F(m) = \alpha \cdot \text{age}(m) - \beta \cdot \text{use}(m) - \gamma \cdot \text{confidence}(m) + \delta \cdot \text{contradictions}(m). \quad (7)$$

When $F(m)$ exceeds a threshold, the memory item can be removed, downgraded, or marked as uncertain. This prevents the optimizer from becoming overconfident in stale experience.

The weights α , β , γ , and δ are normalization factors that must be calibrated to ensure that the four terms operate at comparable scales. In practice, $\text{age}(m)$, $\text{use}(m)$, $\text{confidence}(m)$, and $\text{contradictions}(m)$ should be normalized to $[0, 1]$ using empirical bounds observed during each run. Without this normalization, the forgetting score could be dominated by whichever raw quantity has the largest numerical scale, which would make the mechanism difficult to reproduce.

4.1.1 Consolidation Policy

Not every short-term item should become long-term knowledge. Consolidation should require repeated support or high contradiction value. Equation (8) gives a simple consolidation rule:

$$C(m) = \lambda_1 \cdot \text{improvement}(m) + \lambda_2 \cdot \text{repeatability}(m) + \lambda_3 \cdot \text{contradiction}(m) - \lambda_4 \cdot \text{noise}(m). \quad (8)$$

If $C(m)$ exceeds a consolidation threshold, the item is converted into or merged with a long-term rule. This policy allows two different kinds of information to be remembered. Repeated high-quality samples are remembered because they are useful. Surprising samples are remembered because they may reveal a false prior. Both are valuable, but for different reasons.

The consolidation policy should also record provenance. A rule learned from one task should not automatically dominate all future tasks. It should carry metadata indicating the source task, dimension, bounds, local region, and the number of times it has been confirmed. Without provenance, the system cannot distinguish durable structure from accidental correlation.

4.1.2 Region Representation

For a first implementation, a memory rule should be simple enough to debug. Equation (9) gives one practical representation as an axis-aligned hyper-rectangle:

$$r = [l_1, u_1] \times [l_2, u_2] \times \cdots \times [l_d, u_d], \quad (9)$$

paired with a label and confidence score. This representation is coarse, but it has three advantages. It is inspectable, it can be updated by merging overlapping intervals, and it can be used directly to bias scout-bee sampling. Later versions may replace hyper-rectangles with clustering, kernel density estimates, surrogate models, or task-conditioned embeddings. The initial scientific question does not require a sophisticated representation; it requires a representation that makes memory transfer and memory correction observable.

4.2 Cognitive Conflict Detection and Hypothesis Generation

The second mechanism is cognitive conflict detection. A conflict occurs when the search outcome is inconsistent with a memory-based expectation or with the progress expected under the current budget. At least three conflict types are central:

1. **High-potential failure:** a region labeled high-potential repeatedly produces low-quality food sources.
2. **Low-potential surprise:** a region labeled low-potential produces unusually high-quality samples or rapid local improvement.
3. **Distributional deviation:** new samples from a region significantly deviate from the historical statistics stored for that region.
4. **Progress stagnation under confident priors:** the algorithm has followed high-confidence memory recommendations, but best-so-far improvement remains below expectation.

Conflict detection is valuable only if it leads to a response. In the HD-MABC, conflicts generate competing hypotheses. For example, a low-potential surprise may produce an endogenous hypothesis and an exogenous hypothesis. The endogenous hypothesis says that the memory is wrong or stale: the region may have been mislabeled in previous tasks. The exogenous hypothesis says that the current task is structurally different: the region is useful only under the present objective or constraint configuration.

Each hypothesis must map to a verifiable subgoal. If the hypothesis is “the historical low-potential region was mislabeled,” the subgoal may be to allocate a small branch colony inside or near that region and compare improvement rate against baseline exploration. If the hypothesis is “the current objective changes the value of this region,” the subgoal may be to sample boundary points, constraint-sensitive points, or task-conditioned variants. A hypothesis that cannot be converted into a search action is not operationally useful.

4.2.1 Conflict Severity

Conflict severity should combine expectation confidence and observation strength. A contradiction against a weak memory rule should not trigger expensive branching. Conversely, a contradiction against a highly confident rule deserves attention because it may reveal a meaningful exception. Equation (10) defines one possible severity score:

$$S = \text{confidence}(m) \cdot \text{deviation}(m, x) \cdot \text{relevance}(x), \quad (10)$$

where m is the memory rule and x is the new sample or sample batch. Here $\text{confidence}(m) \in [0, 1]$, $\text{deviation}(m, x) \in [0, 1]$, and $\text{relevance}(x) \in [0, 1]$ are each normalized to the unit interval so that the three factors contribute at comparable scales. A branch event is triggered only when $S > S_{\min}$. This threshold prevents the algorithm from treating ordinary stochastic variation as a cognitive crisis.

The severity score also supports prioritization. If multiple conflicts are detected simultaneously, the decision engine can branch on the conflict with the highest expected value of information. This keeps the mechanism compatible with limited evaluation budgets.

4.2.2 Hypothesis Templates

The first prototype can use a small library of hypothesis templates rather than open-ended reasoning. Examples include:

1. **Stale-memory hypothesis:** a long-term rule was valid in earlier tasks but no longer applies. The test is to sample inside the contradicted region with reduced memory penalty.
2. **Task-exception hypothesis:** the current task is an exception to the task-family rule. The test is to explore the region only under the current task context and avoid immediately overwriting global memory.
3. **Sampling-noise hypothesis:** the surprising observation is an outlier. The test is to resample nearby points and require repeated support before modifying memory.
4. **Resolution hypothesis:** the region descriptor is too coarse. The test is to split the region into subregions and update confidence separately.

These templates keep hypothesis generation operational. They also make later ablation easier: one can remove individual templates and test whether performance changes.

4.3 Bounded Asynchronous Branch Exploration

The third mechanism is bounded asynchronous branch exploration. When multiple hypotheses are generated, the optimizer should not fully commit to one immediately. It should create several search branches, each associated with a hypothesis, and give each branch a limited evaluation budget. This implements a practical rule: do not stay too long in one explanatory story without evidence.

A branch contains a local population, an initialization rule, a branch objective, a maximum evaluation count, and a reporting metric. Branches may run asynchronously because different regions can require different amounts of time per evaluation, especially when objective functions are expensive. The main colony continues to maintain the best-so-far solution and can terminate weak branches early.

At the end of branch testing, the decision engine compares branch outcomes. A branch can be accepted if it produces a substantially better solution, increases the confidence of a useful memory correction, or explains a conflict more parsimoniously than competing hypotheses. A branch can be rejected if it fails to improve, consumes its budget without evidence, or duplicates an already tested region.

The mechanism is deliberately budget-limited. The purpose is not exhaustive exploration of every hypothesis. The purpose is rapid, evidence-seeking correction when memory and observation disagree.

4.3.1 Branch Arbitration

Branch arbitration should compare both solution quality and explanatory value. A branch that produces a slightly better solution but does not resolve the conflict may be less useful than a branch that strongly invalidates a harmful memory rule. Equation (11) defines a practical branch score:

$$B(h) = \eta_1 \cdot \Delta f(h) + \eta_2 \cdot \Delta \text{confidence}(h) + \eta_3 \cdot \text{conflict_resolved}(h) - \eta_4 \cdot \text{budget}(h), \quad (11)$$

where h is the tested hypothesis. The winning branch is the one with the largest $B(h)$, provided it exceeds a minimum acceptance threshold. If no branch passes the threshold, the algorithm returns to the main search and downgrades the confidence of the conflicting rule rather than replacing it outright.

4.3.2 Asynchrony and Reproducibility

Asynchronous branch execution is attractive because branches can be evaluated independently. However, uncontrolled asynchrony can harm reproducibility. A first implementation should use deterministic random seeds per branch, fixed branch budgets, and a clear merge rule. If Python threads are used, objective evaluation should be treated as the unit of accounting: every branch report must state exactly how many evaluations it consumed. This prevents a faster branch from receiving an unfair advantage merely because it ran more iterations.

4.4 Minimal Operational Loop

A minimal implementation can follow the operational loop in Table 3. The loop is written as implementation-oriented pseudocode rather than final algorithmic code, because the exact memory representation and branch policy remain experimental design choices.

This loop keeps the first implementation feasible. It requires no deep learning framework and can be built with standard numerical tools.

Table 3: Minimal operational loop for HD-MABC.

Step	Operation
1	Receive task $T_k = (f_k, \Omega_k, B_k)$; retrieve relevant long-term memory rules.
2	Initialize the bee colony using a mixture of random samples and memory-biased samples.
3	Run employed-bee and onlooker-bee updates using the standard ABC perturbation rule.
4	Label each new sample in the short-term buffer as positive, negative, noisy, or contradictory evidence.
5	Update best-so-far solution and apply scout-bee replacement when food-source trial limits are exceeded.
6	Periodically consolidate reliable short-term evidence into long-term memory.
7	Detect cognitive conflict by comparing current evidence with activated memory priors.
8	If conflict severity exceeds a threshold, generate hypothesis templates and create bounded branch colonies.
9	Execute branches with fixed budgets; compare branch reports using the branch arbitration score.
10	Merge accepted branch outcomes into the main colony; downgrade, split, delete, or revise contradicted memory rules.
11	Apply forgetting to stale or repeatedly contradicted memory items; continue until B_k is exhausted.

4.5 Prototype Parameter Set

The first implementation should expose only a small number of parameters. Too many parameters would make the later empirical claim difficult to interpret. Table 4 lists a conservative initial set.

Future studies should report sensitivity to M , $S_{\min}$, B_{branch} , and p_m , because these parameters directly control the tradeoff between exploitation of memory and skepticism toward memory.

5 Scientific Conjectures

This section states two falsifiable conjectures. They are not conclusions. They are predictions that future experiments should attempt to validate or refute.

5.1 Conjecture 1: Memory Acceleration Effect

In a long sequence of related optimization tasks, an optimizer with selective memory should require fewer function evaluations to reach a target error threshold as the number of observed tasks increases. More formally, let $E_{a,k}$ be the number of evaluations required by algorithm a to reach a predefined target error on task k . For a memory-augmented algorithm a_m , the expected value $\mathbb{E}[E_{a_m,k}]$ should decrease across the sequence when tasks share reusable structure. For a memoryless baseline a_0 , $\mathbb{E}[E_{a_0,k}]$ should remain approximately flat or fluctuate without a consistent learning trend.

The expected empirical signature is a downward curve for the memory-augmented optimizer over sequential tasks, compared with a relatively horizontal curve for standard ABC. The key measurement should be evaluations-to-target rather than final best value alone, because the conjecture concerns learning efficiency across tasks.

This conjecture should be tested with care. The target error threshold must be defined before running the experiment. If the global optimum is known, the threshold can be based on distance

Table 4: Suggested parameters for a minimal prototype.

Parameter	Suggested role	Reason for inclusion
Colony size N	ABC population size	Controls baseline search capacity.
Trial limit L	Scout trigger	Standard ABC abandonment mechanism.
Memory capacity M	Long-term rule limit	Prevents unbounded archive growth.
Forgetting threshold $F_{\min}$	Rule deletion or downgrade	Removes stale or disproven associations.
Conflict threshold $S_{\min}$	Branch trigger	Prevents overreaction to noise.
Branch budget B_{branch}	Hypothesis test cost	Makes re-exploration bounded and comparable.
Memory bias probability p_m	Initialization/scout bias	Controls how strongly prior memory influences search.

to the known optimum. If the optimum is not known, the threshold should use a best-known reference value and should be reported honestly. The task sequence must also be designed so that transfer is possible but not trivial; otherwise, the experiment would only show that a hand-coded bias matches a hand-designed benchmark.

5.1.1 Falsification Condition

If the memory-augmented optimizer does not exhibit a statistically significant downward trend in evaluations-to-target across the task sequence, Conjecture 1 is unsupported. In particular, if a Mann–Kendall trend test applied to evaluations-to-target across tasks fails to reject the null hypothesis of no trend at $p < 0.05$, the conjecture should be considered empirically unsubstantiated. The conjecture should also be considered unsupported if the same downward trend appears in the memoryless baseline under the same task sequence, because that would indicate that the effect is caused by task ordering or benchmark design rather than memory transfer.

5.1.2 Future Experimental Protocol

A minimal test should use a sequence of eight related continuous optimization tasks. Each task should have known bounds and a known or reproducible target value. For each algorithm and each task, the experiment should record the number of function evaluations needed to reach a fixed error threshold ϵ . If a run fails to reach the threshold within the budget, it should be counted as censored at the maximum budget rather than silently discarded. At least 30 independent runs should be used per task, with fixed random seed schedules shared across algorithms.

The memory-augmented algorithm should carry long-term memory from task t to task $t + 1$. The standard ABC baseline should restart independently for each task. A stronger evaluation should include at least two additional baselines: an ABC variant with elite-solution memory but no conflict detection, and a memory-augmented variant with conflict detection disabled. This separates the value of ordinary memory reuse from the value of conflict-aware memory revision.

5.2 Conjecture 2: Hypothesis Leap Effect

In a deceptive optimization landscape where the global optimum lies in a region that long-term memory incorrectly marks as low-potential, an optimizer with cognitive conflict detection and hypothesis-driven branch exploration should eventually revise its prior and redirect search into

the penalized region. The expected convergence curve should contain a characteristic sharp drop in best-so-far error after conflict detection and branch testing.

The baseline prediction is different. A standard memoryless ABC may still find the global optimum by chance, but it lacks a mechanism that explicitly detects conflict between prior expectation and observed stagnation. A memory-augmented optimizer without conflict detection may be worse: it may obey the false prior and avoid the correct region. The hypothesis leap effect is present only if the algorithm can both use memory and question memory.

5.2.1 Falsification Condition

If the memory-augmented optimizer does not produce a statistically identifiable post-conflict drop in $\log(\epsilon_{\text{best}}(n))$, Conjecture 2 is unsupported. In particular, if a change-point test or a pre/post conflict slope comparison fails to reject the null hypothesis of no post-conflict acceleration at $p < 0.05$, the conjecture should be considered empirically unsubstantiated. The conjecture should also be treated as unsupported if comparable cliff-like drops occur in standard ABC at similar rates without conflict events, because that would indicate ordinary stochastic discovery rather than a hypothesis-driven leap.

5.2.2 Future Experimental Protocol

A minimal deceptive-landscape test should intentionally plant a false prior in long-term memory. For example, before optimization begins, the memory base can mark a quadrant or hyper-rectangular region as low-potential. The synthetic objective should then place the global optimum inside that region while placing one or more broad local optima outside it. The evaluation metric should be $\log(\epsilon_{\text{best}}(n))$, where $\epsilon_{\text{best}}(n)$ is the best-so-far error after n evaluations.

The expected signature is not merely a better final solution. The distinctive signature is temporal: the memory-augmented algorithm should initially behave as if the false prior is plausible, then detect conflict after stagnation or surprising evidence, then allocate a branch that re-enters the penalized region, and finally produce a sharp reduction in best-so-far error. The experiment should log the evaluation index at which conflict is detected, the hypotheses generated, branch budgets consumed, and the accepted branch. Without this trace, the observed drop could be misinterpreted as ordinary stochastic discovery.

5.3 Possible Negative Results

Both conjectures can fail, and those failures would be scientifically useful. The memory acceleration effect may fail if the task sequence lacks reusable structure, if the memory representation is too coarse, or if the memory confidence update overfits early tasks. The hypothesis leap effect may fail if conflict detection triggers too late, if branch budgets are too small, or if the deceptive optimum is too narrow for any branch to sample reliably.

These failure modes should not be hidden. They define the boundary conditions of the framework. A mature evaluation should report when memory helps, when it is neutral, and when it harms search. The most important empirical question is not whether memory always improves ABC. It is whether selective, conflict-aware memory improves the reliability of optimization across related but partially deceptive tasks.

5.4 Ablation Plan

The two conjectures require ablation tests. At minimum, a future implementation should compare four variants:

1. **Standard ABC:** no long-term memory and no conflict-driven branching.

2. **ABC + memory only:** long-term memory biases initialization or scout behavior, but conflict detection is disabled.
3. **ABC + conflict only:** conflict detection is enabled inside a task, but no cross-task long-term memory is retained.
4. **Full HD-MABC:** selective memory, forgetting, conflict detection, hypothesis generation, and bounded branch exploration are all enabled.

This ablation plan is necessary because the framework combines several mechanisms. If the full method improves performance but the memory-only variant performs similarly, hypothesis branching may not be necessary. If the memory-only variant performs poorly on deceptive tasks while the full method recovers, that would support the claim that the algorithm benefits from questioning its own memory.

6 Conclusion and Future Work

This report introduced the Hypothesis-Driven Memory-Augmented Bee Colony Framework, a conceptual architecture that maps selective memory, conflict detection, and hypothesis verification into a bee-colony optimization framework. The central contribution is the framing of swarm optimization as a lifelong, memory-managed process rather than a sequence of isolated searches.

The framework makes three claims at the architectural level. First, historical search experience should be represented as structured, revisable knowledge rather than as an undifferentiated archive. Second, an optimizer should treat memory as fallible: when observation contradicts expectation, the algorithm should generate and test alternative explanations. Third, branch exploration should be bounded so that curiosity does not become uncontrolled evaluation spending.

The immediate next step is implementation. A first prototype should implement standard ABC, a short-term memory buffer, an interval-based long-term memory store, conflict-triggered hypothesis generation, and bounded branch exploration. The first empirical target should be two small experiments: a sequential task stream for the memory acceleration effect and a deceptive landscape for the hypothesis leap effect. Later versions should add stronger baselines, parameter sensitivity analysis, real-world optimization tasks, and public replication scripts.

6.1 Limitations

Several limitations should be stated explicitly. The current report is architectural and conjectural; it does not provide experimental validation. The biological analogy is an engineering metaphor, not a neuroscientific model. The simplest interval-based memory representation may not scale well to high-dimensional spaces. Branch exploration consumes additional evaluations and may reduce performance when conflicts are noisy or uninformative. Finally, the framework may be most useful for streams of related tasks; on independent tasks with no reusable structure, memory can offer little benefit and may be harmful.

6.2 Roadmap

The planned research sequence is:

1. implement a clean standard ABC baseline;
2. add short-term evidence labeling and interval-based long-term memory;
3. implement conflict detection and hypothesis-template generation;
4. implement bounded branch colonies with deterministic budgets and logs;

5. run the two conjecture-driven experiments with 30 independent runs;
6. publish code, random seeds, raw logs, and plotting scripts;
7. update this report with empirical results only after the results are reproducible.

If the conjectures are supported, the next scientific step will be to compare the framework against memory-based evolutionary algorithms and dynamic-environment methods under a broader benchmark suite. If the conjectures are not supported, the failure cases should be analyzed to determine whether the issue lies in the memory representation, conflict trigger, branch budget, or the conjectures themselves.

References

- [1] Alan Baddeley. The episodic buffer: A new component of working memory? *Trends in Cognitive Sciences*, 4(11):417–423, 2000. doi: 10.1016/S1364-6613(00)01538-2.
- [2] Matthew M. Botvinick, Todd S. Braver, Deanna M. Barch, Cameron S. Carter, and Jonathan D. Cohen. Conflict monitoring and cognitive control. *Psychological Review*, 108(3):624–652, 2001. doi: 10.1037/0033-295X.108.3.624.
- [3] Jürgen Branke. Memory enhanced evolutionary algorithms for changing optimization problems. In *Proceedings of the 1999 Congress on Evolutionary Computation*, volume 3, pages 1875–1882, 1999.
- [4] Jürgen Branke. *Evolutionary Optimization in Dynamic Environments*. Kluwer Academic Publishers, 2002.
- [5] Marco Dorigo, Vittorio Maniezzo, and Alberto Coloni. Ant system: Optimization by a colony of cooperating agents. *IEEE Transactions on Systems, Man, and Cybernetics, Part B*, 26(1):29–41, 1996. doi: 10.1109/3477.484436.
- [6] John H. Holland. *Adaptation in Natural and Artificial Systems*. University of Michigan Press, 1975.
- [7] Yaochu Jin and Jürgen Branke. Evolutionary optimization in uncertain environments—a survey. *IEEE Transactions on Evolutionary Computation*, 9(3):303–317, 2005. doi: 10.1109/TEVC.2005.846356.
- [8] Dervis Karaboga. An idea based on honey bee swarm for numerical optimization. Technical Report TR06, Erciyes University, 2005.
- [9] Dervis Karaboga and Bahriye Basturk. A powerful and efficient algorithm for numerical function optimization: Artificial bee colony (ABC) algorithm. *Journal of Global Optimization*, 39(3):459–471, 2007. doi: 10.1007/s10898-007-9149-x.
- [10] James Kennedy and Russell Eberhart. Particle swarm optimization. In *Proceedings of ICNN’95 – International Conference on Neural Networks*, volume 4, pages 1942–1948, 1995. doi: 10.1109/ICNN.1995.488968.
- [11] Scott Kirkpatrick, C. Daniel Gelatt, and Mario P. Vecchi. Optimization by simulated annealing. *Science*, 220(4598):671–680, 1983. doi: 10.1126/science.220.4598.671.
- [12] Dharshan Kumaran, Demis Hassabis, and James L. McClelland. What learning systems do intelligent agents need? complementary learning systems theory updated. *Trends in Cognitive Sciences*, 20(7):512–534, 2016. doi: 10.1016/j.tics.2016.05.004.

- [13] James L. McClelland, Bruce L. McNaughton, and Randall C. O'Reilly. Why there are complementary learning systems in the hippocampus and neocortex: Insights from the successes and failures of connectionist models of learning and memory. *Psychological Review*, 102(3):419–457, 1995. doi: 10.1037/0033-295X.102.3.419.
- [14] Trung Thanh Nguyen, Shengxiang Yang, and Jürgen Branke. Evolutionary dynamic optimization: A survey of the state of the art. *Swarm and Evolutionary Computation*, 6: 1–24, 2012. doi: 10.1016/j.swevo.2012.05.001.
- [15] Rainer Storn and Kenneth Price. Differential evolution – a simple and efficient heuristic for global optimization over continuous spaces. *Journal of Global Optimization*, 11(4):341–359, 1997. doi: 10.1023/A:1008202821328.
- [16] Shengxiang Yang. Genetic algorithms with memory- and elitism-based immigrants in dynamic environments. *Evolutionary Computation*, 16(3):385–416, 2008. doi: 10.1162/evco.2008.16.3.385.